

# torch.Tensor.index\_reduce\_#

[https://pytorch.org/docs/stable/generated/torch.Tensor.index\\_reduce\\_.html#to](https://pytorch.org/docs/stable/generated/torch.Tensor.index_reduce_.html#torch.Tensor.index_reduce_)

## Description:

Accumulate the elements of source into the self tensor by accumulating to the indices in the order given in index using the reduction given by the reduce argument. For example, if `dim == 0`, `index[i] == j`, `reduce == prod` and `include_self == True` then the *i*th row of source is multiplied by the *j*th row of self. If `include_self="True"`, the values in the self tensor are included in the reduction, otherwise, rows in the self tensor that are accumulated to are treated as if they were filled with the reduction identities. The *dim*th dimension of source must have the same size as the length of index (which must be a vector), and all other dimensions must match self, or an error will be raised. For a 3-D tensor with `reduce="prod"` and `include_self=True` the output is given as: This operation may behave nondeterministically when given tensors on a CUDA device. See [Reproducibility](#) for more information.

## Function Signature

---

```
Tensor.index_reduce_(dim, index, source, reduce, *,  
include_self=True) → Tensor#
```

Parameters

Keyword Arguments

## Parameters

---

### Keyword

`include_self` (bool) – whether the elements from the self tensor are included in the reduction

## Code Examples

---

```
self[index[i], :, :] *= src[i, :, :] # if dim == 0
self[:, index[i], :] *= src[:, i, :] # if dim == 1
self[:, :, index[i]] *= src[:, :, i] # if dim == 2
```

```
>>> x = torch.empty(5, 3).fill_(2)
>>> t = torch.tensor([[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12]], dtype=torch.float)
>>> index = torch.tensor([0, 4, 2, 0])
>>> x.index_reduce_(0, index, t, 'prod')
tensor([[20., 44., 72.],
        [ 2.,  2.,  2.],
        [14., 16., 18.],
        [ 2.,  2.,  2.],
        [ 8., 10., 12.]])
>>> x = torch.empty(5, 3).fill_(2)
>>> x.index_reduce_(0, index, t, 'prod', include_self=False)
tensor([[10., 22., 36.],
        [ 2.,  2.,  2.],
        [ 7.,  8.,  9.],
        [ 2.,  2.,  2.],
        [ 4.,  5.,  6.]])
```

## Notes & Warnings

---

### NOTE

Note This operation may behave nondeterministically when given tensors on a CUDA device. See Reproducibility for more information.

### NOTE

Note This function only supports floating point tensors.



### WARNING

Warning This function is in beta and may change in the near future.

## Detailed Documentation

---

### Documentation

Accumulate the elements of source into the self tensor by accumulating to the indices in the order given in index using the reduction given by the reduce argument. For example, if `dim == 0`, `index[i] == j`, `reduce == prod` and `include_self == True` then the *i*th row of source is multiplied by the *j*th row of self. If `include_self="True"`, the values in the self tensor are included in the reduction, otherwise, rows in the self tensor that are accumulated to are treated as if they were filled with the reduction identities.

The `dim`th dimension of source must have the same size as the length of index (which must be a vector), and all other dimensions must match self, or an error will be raised.

For a 3-D tensor with `reduce="prod"` and `include_self=True` the output is given as:

This operation may behave nondeterministically when given tensors on a CUDA device. See [Reproducibility](#) for more information.

This function only supports floating point tensors.

This function is in beta and may change in the near future.

`dim (int)` – dimension along which to index

`index (Tensor)` – indices of source to select from, should have dtype either `torch.int64` or `torch.int32`

`source (FloatTensor)` – the tensor containing values to accumulate

`reduce (str)` – the reduction operation to apply ("prod", "mean", "amax", "amin")

`include_self (bool)` – whether the elements from the self tensor are included in the reduction